Contents

Threat Register **1**
- Status . . . 1
- Legend . . . 1
- Register . . . 1
- Explicitly out of scope . . . 10
- Maintenance . . . 10
- Related . . . 10

Threat Register

Subject V.6 requires more than protections: *"you must identify other dangers and describe the protection measures that can be implemented"*. This is that document.

A written accepted risk is an engineering decision. An unwritten one is an oversight. Evaluators ask what you did *not* protect against; this table is the answer.

Status

Design-stage register. 26 threats. Slice S0 landed the first verified rows, S1 added T-25 and T-26; every other mitigation below is *designed*, not verified. The Verified column stays empty until a test, a config artifact or a named code path proves it.

Legend

Likelihood and Impact: L / M / H. Residual = what remains **after** the planned mitigation.

Register

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-01	Credential stuffing / brute force on login	H	H	rate limit per account **and** per IP, progressive backoff, generic errors, BCrypt cost >= 12	distributed low-and-slow attempts across many IPs remain possible; no breached-password feed at launch	`UNKNOWN - requires decision`	—
T-02	Session hijacking via stolen access token	M	H	15-minute access token, HTTPS only, no token in URLs or logs	a stolen token is valid until expiry; no device binding		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-03	Refresh-token theft	M	H	opaque, hashed at rest, rotated on every use, family revoked on reuse detection	detection is after the fact, not preventive		—
T-04	Broken object-level authorization (another user's data)	M	H	two gates in a pipeline behaviour, checked on every request and hub join; hidden resources return `404`	a new endpoint that bypasses the behaviour; mitigated by a required negative test per endpoint		—
T-05	Vote manipulation (double voting, stuffing)	H	M	unique index `(EventId, TrackId, UserId)` as the arbiter, rate limit on the vote endpoint	multiple accounts by one person (sybil) is not prevented; email verification raises the cost only		—
T-06	Playlist reorder conflict causing lost or divergent order	H	M	sparse ordering keys plus concurrency token, reload-and-reapply or `409`	rapid concurrent edits still surface as `409` to the user		—
T-07	Account takeover through unverified provider email matching	L	H	match an existing account only when the provider asserts `email_verified`	a provider that wrongly asserts verification		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-08	ID token substitution (token minted for another app)	L	H	validate `aud`, `iss`, `exp`, `nonce` against JWKS, server-side	depends on correct JWKS caching and key rotation handling		—
T-09	API key or secret committed to git	L	**critical**	`.env` git-ignored from commit 1, `.env.example` placeholders only, rotate-first leak procedure	subject V.8 makes this an **automatic project failure**; process discipline is the only control		S0: `MusicRoom/.dockerignore` excludes `.env`; no `.env` and no credential string in any layer of the built image; `docker-compose.yml` carries no literal
T-10	YouTube quota exhaustion during the demo	M	M	prefer `videos.list` over `search.list`, debounce, cache `TrackRef`, handle `403 quotaExceeded` explicitly	a determined user can still burn quota; no per-user quota budget at launch		—
T-11	Denial of service via unbounded input or list requests	M	M	capped `pageSize`, bounded string and collection lengths, global plus per-endpoint rate limits	a single instance has no upstream protection; capacity is whatever V.7 measures		—
T-12	SignalR connection exhaustion	M	M	bounded payloads, explicit keep-alive and timeout settings	no backplane (issue O8), so one instance is the ceiling		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-13	Database connection-pool exhaustion	M	M	small pool, short-lived connections, pooled connection string for the API	Supabase enforces a hard connection ceiling above ours		—
T-14	Personal data over-exposure in profile responses	M	M	per-field filtering server-side by requester relationship	a new profile field added without a visibility tier; mitigated by making the tier non-nullable		—
T-15	Sensitive data in logs	M	H	never-logged list, structured templates, no full bodies on credential endpoints	a developer logging an object graph that contains a token		partial, S0: `GetHealthReady_WhenData` proves the readiness body carries no host, connection string or exception text
T-16	Vulnerable transitive dependency	M	M	`dotnet list package --vulnerable --include-transitive` fails CI on high severity	a zero-day between scans		S0: step present in both workflows and in `make agent-check`; clean across all 10 projects

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-17	Location+time licence bypass (spoofed GPS)	M	M	server-side geofence check **plus** an optional short-lived event code shown at the venue; the licence policy selects which proofs are required — [[ADR-0014-location-time-licence]]	client-reported coordinates remain spoofable; the event code raises the cost but a code can be relayed to someone off-site	`UNKNOWN - requires decision`	—
T-18	Malicious content in user-supplied text (event names, playlist titles)	M	M	validation, length caps, output encoding at the consumer	the client is responsible for rendering safely; no web frontend today		—
T-19	Mail provider quota exhausted by a registration loop	L	M	rate limits on registration and verification resend; hosted provider free tier — [[ADR-0013-mail-provider]]	a quota exhausted mid-defence blocks the verification demo		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-20	Supabase free project pauses from inactivity	M	H	touch the project before the defence; local PostgreSQL container as a fallback with identical migrations — [[ADR-0016-supabase-plan-region]]	a pause discovered on the day still costs demo time		—
T-22	Malicious controller skips the queue	L	M	only `controllerUserId` may advance; `expectedNowPlayingId` makes a replay idempotent; the owner can revoke control at any time	a legitimate controller can still play ahead — that is what control means		—
T-23	Playback stalls because the controller device went offline	M	M	control takeover via `PUT /devices/{id}/control`; the owner can revoke	by design; no server-side watchdog ([[ADR-0017-playback-session]]); recovery is manual (issue O13)		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-24	Operational endpoints reachable by anyone who can reach the port	M	L	`/health` and `/health/ready` are anonymous by contract ([[endpoint-inventory]]) and report only check names and states — no version, host, connection string or exception text	an unauthenticated caller learns the service is up and whether its database is reachable, and can poll it: the endpoints carry no rate-limit policy, because a limiter that throttles a probe defeats the probe		S0: readiness body asserted by test; no authentication is planned for these two

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-25	Anonymous auth endpoints have no rate limit: `POST /api/v1/auth/registrations` and `POST /api/v1/auth/email-verifications` can both be driven in a loop	M	M	rate limits keyed on IP and on the submitted address (SDD §13.3), built once for registration, verification, login, reset and resend together	**unmitigated today.** S1 shipped both endpoints without a limiter, deliberately: the control is shared with endpoints that do not exist yet, and building it per endpoint means building it four times. Until it lands, an unauthenticated caller can create rows at request rate, can burn the mail provider quota described in T-19 once O17 lands a real sender, and can guess verification tokens at request rate. The `202`-on-both-paths behaviour on registration, and the single `400` for unknown, malformed and spent tokens on verification, mean	accepted for S1 by SPEC-002, surface widened by SPEC-003; **must** be closed before the defence, V.6 grades it	—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-26	Development-only `LoggingMailService` writes the email verification link, token included, to the application log	M	M	registered as `IMailService` only when the host environment is Development; `AddInfrastructureServices` throws `InvalidOperationException` at start-up in any other environment rather than falling back silently or booting with no sender. With no `VERIFICATION_LINK_BASE_URL` configured the sender logs the bare token and says the variable is missing — it never invents a default link	**accepted.** Anyone who can read a developer's container logs can verify an account created on that machine. The exposure is bounded to Development hosts by a mechanical guard, not a convention, and a production host fails loudly instead of leaking. The value is that the O17 gap does not block the flow: a developer reads the link out of `docker compose logs` and exercises verification end to end before a mail provider exists. Removed when O17 lands the hosted provider (slice S1a), at which point this row closes	owner, 2026-09-26, on SPEC-003	S1: `LoggingMailServiceTests` asserts one event carrying the link, and the non-Development start-up throw is a named code path in `InfrastructureServiceRe`

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-21	Voter location data stored unnecessarily	L	M	the submitted coordinate is used for the check and **not** persisted on the vote row; event centre stored at coarse precision	the coordinate exists in transit and in request logs unless excluded — add it to the never-logged list		—

Explicitly out of scope

Not defended against	Why
A determined attacker with unlimited resources	subject V.6: *"your API is not expected to be unbreakable"*
Physical device compromise	the token is in the platform secure store; a rooted device is outside our control
Supabase platform compromise	vendor trust boundary, stated rather than pretended away
Sybil accounts (T-05)	email verification raises cost; full prevention needs identity proofing this project will not build

Maintenance

- A finding from [[owasp-top10-checklist]] or [[owasp-api-top10-checklist]] that is not fixed in the pull request gets a row here **before** that PR merges.
- Fill Accepted by with a real name when the team agrees to carry a residual risk.
- Fill Verified with the test or artifact that proves the mitigation, once it exists.
- Never delete a row. Close it by pointing at the evidence.

Related

[[owasp-index]] · [[asvs-conformance]] · [[application-security]] · [[authentication-authorization]] · [[sdd]] §13.6 · [[subject-requirements]]